Aula 17 — Inventário de jogo

Interfaces e capacidades

Como guardar itens diferentes e permitir operações específicas sem exigir que todos ofereçam as mesmas capacidades?

DE VOLTA DA EXPLORAÇÃO

Três itens chegam ao inventário

Item	Estado inicial	Reparo com uma unidade
Espada de ferro	`40/100`	ferro: `+25`, até `100`
Escudo	`30/80`	madeira: `+20`, até `80`
Maçã	sem durabilidade	não aceita reparo

O **inventário** guarda e apresenta os itens. A **bigorna** tenta reparar o item selecionado com uma pilha de material.

AS REGRAS DESTE MODELO

Uma tentativa também envolve o material

Sucesso: devolve `true`, consome uma unidade e aumenta a durabilidade até o máximo.

Recusa: devolve `false` e preserva item e pilha — material incompatível, pilha vazia, equipamento completo ou maçã.

Mesmo uma espada em `90/100` consome uma unidade inteira para chegar a `100/100`.

Um contrato comum basta para descrever

```
public interface Item {
    String descrever();
}
```

```
public class Maca implements Item {
    public String descrever() {
        return "Maçã";
    }
}
```

`Espada` e `Escudo` também cumprem `Item`; suas descrições incluem a durabilidade.

Guardar e consultar referências de Item

```
private List<Item> itens; // inicializada com new ArrayList<>()

public Item obter(int indice) {
    return itens.get(indice);
}
public void listar() {
    for (Item item : itens) {
        System.out.println(item.descrever());
    }
}
```

`adicionar(Item item)` guarda a referência recebida. `obter(...)` consulta uma posição, sem remover nem copiar o item.

ATIVIDADE

CENÁRIO-BASE — AINDA SEM REPAROS

O que aparece na listagem?

```
Inventario inventario = new Inventario();
inventario.adicionar(new Espada(40)); // posição 0
inventario.adicionar(new Escudo(30)); // posição 1
inventario.adicionar(new Maca());     // posição 2
PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
inventario.listar();
```

1. Quais itens são apresentados, em que ordem e com quais durabilidades?
2. No laço, `item` tem que tipo declarado? Precisamos testar a classe para escolher `descrever()`?

O contrato permite; cada objeto responde

```
Espada de ferro: 40/100
Escudo: 30/80
Maçã
```

O tipo declarado de `item` é **Item**. A chamada `descrever()` já pertence a esse contrato.

Não precisamos testar classes: o **despacho dinâmico** escolhe a implementação do objeto alcançado em cada passagem.

LISTAR FUNCIONOU; AGORA O JOGADOR SELECIONA A ESPADA

A chamada de reparo passa a ser permitida?

Nesta versão, `Espada` e `Escudo` implementam somente `Item`, mas já possuem métodos públicos `repararCom(...)` e `getDurabilidade()`.

```
Item selecionado = inventario.obter(0);
selecionado.repararCom(ferro); // não compila
```

O objeto é uma espada. Por que o compilador recusa a chamada?

O compilador consulta o tipo declarado

```
public interface Item {
    String descrever();
}
```

`Item` não declara `repararCom(...)`. Essa chamada é recusada mesmo que o objeto selecionado possua o método.

O despacho dinâmico escolhe a implementação de uma **chamada aceita**; ele não acrescenta operações ao contrato na compilação.

A pilha cuida de sua quantidade

Operação de PilhaMaterial	O que oferece
`ehDoTipo("ferro")`	confere o tipo do material
`consumirUma()`	retira uma unidade se houver saldo e devolve `true`; senão, devolve `false`
`getQuantidade()`	consulta o saldo

O equipamento pode solicitar consumo; a pilha impede que sua quantidade fique negativa.

Só reparar depois de conseguir o material

```
public boolean repararCom(PilhaMaterial material) {
    if (!material.ehDoTipo("ferro") || durabilidade == 100) {
        return false;
    }
    if (!material.consumirUma()) {
        return false;
    }
    // atualização da durabilidade no próximo slide
}
```

Material incompatível, espada completa ou pilha vazia encerram a tentativa sem aumentar a durabilidade.

CONTINUAÇÃO DO MESMO MÉTODO — UMA UNIDADE JÁ FOI CONSUMIDA

A espada protege seu limite

```
durabilidade += 25;
if (durabilidade > 100) {
    durabilidade = 100;
}
return true;
```

`40 → 65`; `90 → 100`.

O escudo segue sua própria regra: aceita madeira e acrescenta `20`, até `80`.

JAVA EM FOCO

VOLTAR À SELEÇÃO DA POSIÇÃO 0 — SABEMOS QUE É UMA ESPADA

Cast: usar a referência por outro tipo

```
Item selecionado = inventario.obter(0);
Espada espada = (Espada) selecionado;
```

`(Espada)` é uma **conversão explícita**, chamada de *cast*. Ela permite usar a referência como `Espada` se o objeto alcançado for compatível.

O cast não executa `new`, não copia o objeto e não altera sua classe ou seu estado.

ATIVIDADE

CENÁRIO-BASE — ESPADA 40/100; FERRO COM 2 UNIDADES

Preveja o efeito do primeiro reparo

```
Item selecionado = inventario.obter(0);
Espada espada = (Espada) selecionado;
boolean reparou = espada.repararCom(ferro);
System.out.println(reparou);
System.out.println(espada.getDurabilidade());
System.out.println(ferro.getQuantidade());
System.out.println(selecionado == espada);
inventario.listar();
```

Quais são as **sete linhas** impressas? Justifique a comparação com `==` e a durabilidade exibida na listagem.

MESMO CENÁRIO — O REPARO ALTEROU A ESPADA GUARDADA

Uma espada, observada por mais de um caminho

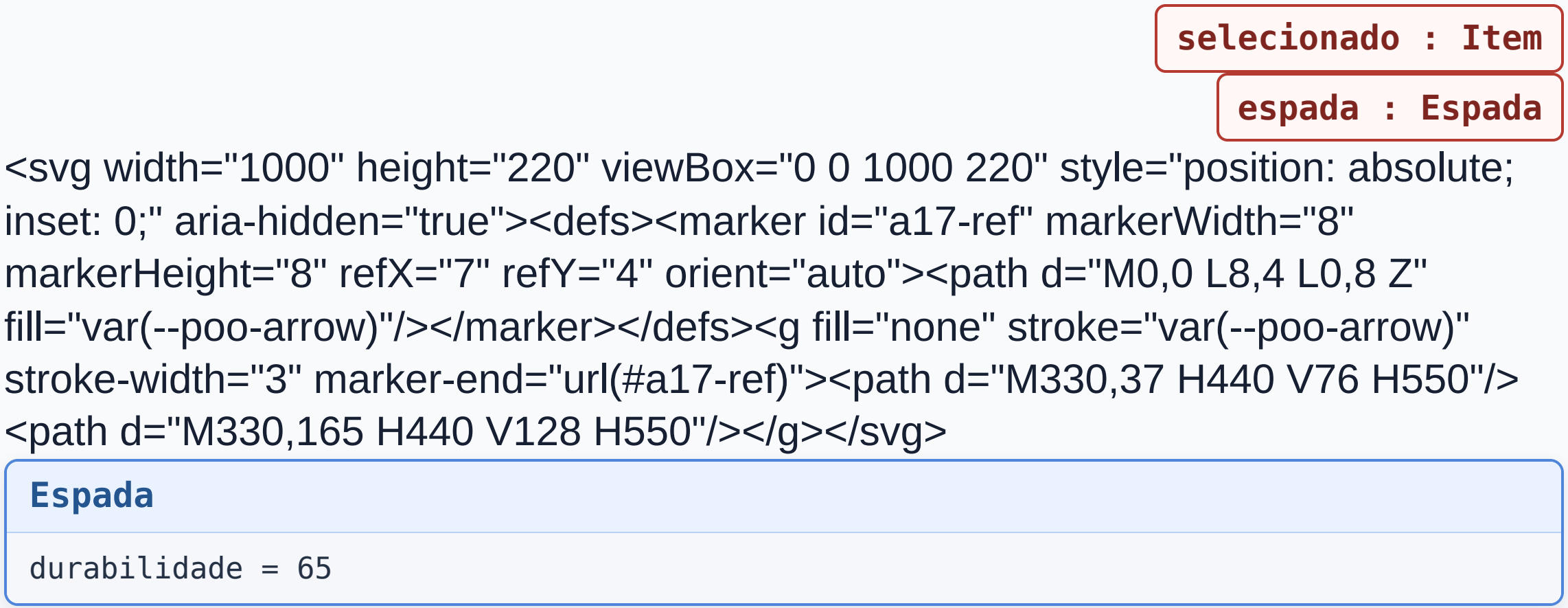
```
true
65
1
true
Espada de ferro: 65/100
Escudo: 30/80
Maçã
```

`==` resulta em `true`: as referências alcançam a mesma espada. A posição `0` do inventário também guarda essa referência.

O cast não muda o tipo de selecionado

`selecionado : Item`

`espada : Espada`



`selecionado` continua declarado como **Item**: `selecionado.repararCom(ferro)` ainda não compila.

ATIVIDADE

CENÁRIO INDEPENDENTE — INVENTÁRIO INICIAL; FERRO NOVO COM 2

E se a seleção alcançar a maçã?

```
Item selecionado = inventario.obter(2);
Espada espada = (Espada) selecionado;
espada.repararCom(ferro);
System.out.println("Terminou");
```

1. O trecho compila? O cast funciona durante a execução?
2. Há consumo de material? A mensagem é impressa?

O cast compila, mas falha na execução

1. **Compila:** o tipo declarado `Item` admite a possibilidade de alcançar uma espada. Ao executar, o objeto real é uma `Maca`: o cast falha com **ClassCastException**.
2. **Não há consumo nem impressão de "Terminou".** Nesse programa sem tratamento da falha, as linhas seguintes não são executadas.

Nenhuma espada foi criada. Precisamos verificar a compatibilidade antes de converter.

instanceof vem antes da conversão

```
Item selecionado = inventario.obter(2);
if (selecionado instanceof Espada) {
    Espada espada = (Espada) selecionado;
    espada.repararCom(ferro);
}
```

`instanceof` produz um `boolean`: testa compatibilidade com o tipo indicado. O ramo executa somente se o resultado for `true`.

Para `null`, o resultado é `false`.

Essa verificação reconhece apenas espadas

Seleção	`instanceof Espada`	Estado após o trecho
espada `40/100`	`true`	espada `65/100`; ferro `1`
escudo `30/80`	`false`	escudo `30/80`; ferro `2`
maçã	`false`	maçã preservada; ferro `2`
`null`	`false`	ramo ignorado; ferro `2`

O escudo também aceita reparo. A pergunta “é uma espada?” é suficiente para a bigorna?

Acrescentar um ramo para cada classe?

```
if (item instanceof Espada) {
    Espada espada = (Espada) item;
    return espada.repararCom(material);
}
if (item instanceof Escudo) {
    Escudo escudo = (Escudo) item;
    return escudo.repararCom(material);
}
```

Um terceiro equipamento exigiria outro ramo, embora o trabalho solicitado continuasse sendo **tentar reparar**.

ATIVIDADE

ESCOLHER O CONTRATO DE QUE A COLABORAÇÃO PRECISA

Como expressar a capacidade de reparo?

Proposta	Decisão
A	exigir `repararCom(...)` de todo `Item`, inclusive da maçã
B	manter um ramo para cada classe concreta na bigorna
C	manter `Item` e criar outro contrato para quem aceita reparo

Qual proposta evita exigir reparo da maçã e listar todas as classes na bigorna? Justifique pelos papéis do inventário e do reparo.

UMA ANÁLISE PELOS PAPÉIS DE CADA CLIENTE

C separa apresentação e capacidade adicional

Inventário: guarda e apresenta qualquer `Item`.

Bigorna: precisa reconhecer quem oferece reparo e solicitar essa operação.

A proposta **A** obriga a maçã a oferecer uma operação fora de seu papel, mesmo que sempre recuse. A proposta **B** vincula a bigorna aos nomes de todas as classes reparáveis.

Reparavel declara a capacidade

```
public interface Reparavel {
    boolean repararCom(PilhaMaterial material);
    int getDurabilidade();
}
```

Os equipamentos já possuem esses métodos públicos. Agora passam a declarar que cumprem também **Reparavel**.

`Maca` continua cumprindo somente `Item`.

implements lista os contratos por vírgula

```
public class Espada implements Item, Reparavel {
    // métodos públicos dos dois contratos
}
```

Tipo declarado da referência	Operações oferecidas pelo contrato
`Item`	`descrever()`
`Reparavel`	`repararCom(...)`, `getDurabilidade()`

`Escudo` também cumpre ambos. O objeto cumprir `Item` não acrescenta `descrever()` a uma referência declarada `Reparavel`.

Verificar, converter e delegar

Dentro de `Bigorna`:

```
public boolean tentarReparar(Item item, PilhaMaterial material) {
    if (item instanceof Reparavel) {
        Reparavel equipamento = (Reparavel) item;
        return equipamento.repararCom(material);
    }
    return false;
}
```

A bigorna usa o item e a pilha recebidos; devolve o resultado da delegação.

Cada mecanismo responde a uma pergunta

Etapa	Pergunta respondida
`item instanceof Reparavel`	o objeto oferece essa capacidade?
`(Reparavel) item`	podemos usar a referência por esse contrato?
`equipamento.repararCom(material)`	qual implementação executa no objeto alcançado?

Na última etapa, o despacho dinâmico escolhe a regra da espada ou do escudo.

Se já recebemos um `Reparavel` não nulo, podemos solicitar o reparo diretamente.

ATIVIDADE

VERSÃO COM REPARAVEL — TRÊS CENÁRIOS INDEPENDENTES

Oferecer reparo garante o sucesso?

Em cada cenário, uma bigorna nova recebe:

Cenário	Item	Pilha
1	espada `100/100`	ferro `2`
2	espada `40/100`	madeira `2`
3	maçã	ferro `2`

1. Para cada cenário, qual o resultado de `instanceof Reparavel`? O método do equipamento executa?
2. Qual o retorno da tentativa e o estado final do item e da pilha em cada cenário?

As três tentativas devolvem false

Cenário	Capacidade	Retorno	Estado final
1 — completa	`true`	`false`	espada `100/100`; ferro `2`
2 — madeira	`true`	`false`	espada `40/100`; madeira `2`
3 — maçã	`false`	`false`	maçã preservada; ferro `2`

Em **1 e 2**, `Espada.repararCom(...)` executa e recusa o reparo. Em **3**, a bigorna não entra no ramo de reparo.

NOVO CENÁRIO — ACOMPANHAR A REGRA DO OUTRO EQUIPAMENTO

O escudo recebe madeira

```
Item item = new Escudo(30);
PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
Bigorna bigorna = new Bigorna();
boolean reparou = bigorna.tentarReparar(item, madeira);
```

A bigorna reconhece `Reparavel` e chama a mesma operação.

Como essa chamada chega ao consumo de madeira e à atualização do escudo?

Cada objeto protege seu estado

1. A **bigorna** verifica a capacidade e delega ao escudo recebido.
2. `Escudo.repararCom(...)` confere material e durabilidade; solicita `consumirUma()` à pilha.
3. A **pilha** reduz seu saldo de `2` para `1` e devolve `true`.
4. O **escudo** aumenta de `30` para `50`, respeitando o máximo `80`, e devolve `true`. A bigorna devolve esse resultado.

A bigorna não calcula `+20` nem `+25` e não cria um item substituto.

ATIVIDADE

VERIFICAR A SOLUÇÃO — ESPADA E MATERIAL NOVOS EM CADA LINHA

O que cada teste precisa observar?

Cenário	Espada inicial	Pilha inicial
Reparo possível	`40/100`	ferro `2`
Material incompatível	`40/100`	madeira `2`
Próximo ao máximo	`90/100`	ferro `2`

Cada cenário executa `bigorna.tentarReparar(espada, material)`.

Preveja o retorno, a durabilidade e a quantidade final de material em cada linha.

EXPECTATIVAS PARA OS TRÊS TESTES PRIORITÁRIOS

Conferir retorno, durabilidade e consumo

Cenário	Retorno	Durabilidade final	Saldo final
Reparo possível	`true`	`65`	ferro `1`
Material incompatível	`false`	`40`	madeira `2`
Próximo ao máximo	`true`	`100`	ferro `1`

No último caso, consome-se uma unidade inteira. O limite impede a durabilidade de chegar a `115`.

O teste guarda acesso à espada original

```
Espada espada = new Espada(40);
Inventario inventario = new Inventario();
inventario.adicionar(espada);
PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
Bigorna bigorna = new Bigorna();

boolean reparou =
        bigorna.tentarReparar(inventario.obter(0), ferro);
```

A tentativa recebe o item do inventário. A variável `espada` permite conferir o estado do objeto fornecido.

As expectativas incluem estado e identidade

```
assertEquals(true, reparou);
assertEquals(65, espada.getDurabilidade());
assertEquals(1, ferro.getQuantidade());
assertEquals(true, inventario.obter(0) == espada);
```

O reparo precisa alcançar **a espada recebida**, consumir material e preservar sua presença no inventário.

Retorno correto, sozinho, não demonstra essa colaboração.

SEGUNDO TESTE — MATERIAL INCOMPATÍVEL DEVE PRESERVAR OS DOIS ESTADOS

A recusa também precisa ser verificada

```
@Test
void materialIncompativelPreservaItemEPilha() {
    Espada espada = new Espada(40);
    PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
    Bigorna bigorna = new Bigorna();
    boolean reparou = bigorna.tentarReparar(espada, madeira);

    assertEquals(false, reparou);
    assertEquals(40, espada.getDurabilidade());
    assertEquals(2, madeira.getQuantidade());
}
```

TERCEIRO TESTE — PERTO DO MÁXIMO, O CONSUMO CONTINUA INTEIRO

O limite pertence à regra do equipamento

```
@Test
void limitaDurabilidadeMesmoConsumindoUmaUnidade() {
    Espada espada = new Espada(90);
    PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
    Bigorna bigorna = new Bigorna();
    boolean reparou = bigorna.tentarReparar(espada, ferro);

    assertEquals(true, reparou);
    assertEquals(100, espada.getDurabilidade());
    assertEquals(1, ferro.getQuantidade());
}
```

ATIVIDADE

OS TRÊS TESTES USAM ESPADAS — SUCESSO, RECUSA POR MATERIAL E LIMITE

Esses testes distinguem quais defeitos?

1. Uma versão repara indevidamente a maçã. Algum dos três testes a detecta?
2. Outra versão repara a espada corretamente, mas nunca consome material. Algum a detecta?

Justifique indicando os objetos fornecidos e as expectativas que passam ou falham.

A COBERTURA DEPENDE DOS CENÁRIOS E DAS OBSERVAÇÕES

O saldo revela um defeito que a durabilidade esconde

1. **Nenhum detecta o reparo da maçã:** os três fornecem espadas. Também não verificam a regra do escudo.
2. **O primeiro e o terceiro detectam a falta de consumo:** esperam saldo `1`, mas observariam `2`, mesmo com durabilidade correta.

Os casos de maçã, escudo, equipamento completo e pilha vazia complementam a cobertura; estão no roteiro para consulta.

ATIVIDADE

EVOLUÇÃO — A SOLUÇÃO CORRETA PASSA A RECEBER UMA PICARETA

Onde a novidade precisa aparecer?

`Picareta` é um item reparável com ferro: acrescenta `25` até `100`, consome uma unidade por sucesso e preserva os estados na recusa.

1. Que contratos e métodos a classe deve oferecer? Onde criá-la e adicioná-la?
2. A listagem e a tentativa da bigorna precisam de novos ramos?
3. Que resultado esperar de um teste com picareta `40/100` e ferro `2`?

A montagem escolhe a nova classe

1. `Picareta implements Item, Reparavel`: oferece `descrever()`, `repararCom(PilhaMaterial)` e `getDurabilidade()`.

```
inventario.adicionar(new Picareta(40));
```

2. **Sem novos ramos:** a listagem usa `Item`; a bigorna verifica `Reparavel` e delega.
3. O teste deve obter **true**, durabilidade **65** e ferro **1**. Recusa e limite complementam a verificação da classe.

Prever e justificar antes de conferir

Quatro decisões: compilação e execução; identidade e estado; contrato e despacho; escolha de cenários de teste.

Produza sua resposta **sem consulta**, antes da discussão.

Atividade formativa, sem nota ou entrega. No estudo com IA, **Nível 1 — Tutor**: primeiro sua previsão; depois ajuda para compreender a dúvida.

ATIVIDADE

INDIVIDUAL 1 — ANALISAR AS ALTERNATIVAS SEPARADAMENTE

Não compila ou falha ao executar?

```
// A
Item item = new Espada(40);
item.repararCom(new PilhaMaterial("ferro", 2));
```

```
// B
Item item = new Maca();
Espada espada = (Espada) item;
```

Classifique A e B e justifique pelo tipo declarado e pelo objeto real.

A para na compilação; B falha na execução

A: `Item` não declara `repararCom(...)`. A chamada não compila, mesmo alcançando uma espada.

B: o cast compila porque uma referência declarada `Item` pode alcançar uma espada. Ao executar, o objeto é uma `Maca`: ocorre **ClassCastException**.

A possibilidade admitida pelo tipo declarado não garante a compatibilidade do objeto encontrado.

ATIVIDADE

INDIVIDUAL 2 — INVENTÁRIO NOVO, SEM REPAROS ANTERIORES

O inventário observa qual estado?

```
Inventario inventario = new Inventario();
inventario.adicionar(new Espada(90));
Item selecionado = inventario.obter(0);
Reparavel equipamento = (Reparavel) selecionado;
PilhaMaterial ferro = new PilhaMaterial("ferro", 2);
equipamento.repararCom(ferro);
System.out.println(inventario.obter(0).descrever());
System.out.println(ferro.getQuantidade());
```

Preveja as duas linhas. Explique por que a atualização aparece no inventário: o cast criou outro equipamento?

O inventário continua alcançando a mesma espada

```
Espada de ferro: 100/100
1
```

O cast não criou outro equipamento. A posição `0`, `selecionado` e `equipamento` alcançam **a mesma espada**.

O reparo altera esse objeto, respeita o máximo `100` e consome uma unidade da pilha recebida.

ATIVIDADE

INDIVIDUAL 3 — ESCUDO E PILHA NOVOS, FORNECIDOS À BIGORNA

O contrato escolhe a regra?

```
Item item = new Escudo(30);
PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
```

1. Por que o cast para `Reparavel` dentro da bigorna é válido?
2. Qual implementação de `repararCom(...)` executa?
3. Qual o retorno, a durabilidade final e o saldo de madeira?
4. Uma referência `Reparavel` permite chamar `descrever()` diretamente? Justifique.

O objeto determina a implementação executada

1. O objeto **Escudo cumpre Reparavel**, condição verificada pelo `instanceof` antes do cast.
2. Executa **Escudo.repararCom(...)**, por despacho dinâmico.
3. Retorno **true**; escudo **50/80**; madeira **1**.
4. **Não.** `descrever()` pertence a `Item`; a referência declarada `Reparavel` não oferece essa operação diretamente.

ATIVIDADE

INDIVIDUAL 4 — DUAS VERSÕES DEFEITUOSAS, ANALISADAS SEPARADAMENTE

Que cenário revela cada defeito?

A: a bigorna cria uma `Espada(40)` e repara essa nova espada, ignorando o item recebido.

B: o equipamento consome uma unidade antes de conferir o material. Se incompatível, devolve `false` sem aumentar a durabilidade.

Para cada defeito, escolha um cenário: informe estado inicial, ação e resultado esperado na solução correta, incluindo saldo. Que observação revelaria a falha?

Observar o item fornecido e o material restante

Versão	Inicial; ação: tentar reparar	Esperado na solução correta	O que revela a falha
A	espada original `40/100`; ferro `2`	`true`; original `65/100`; ferro `1`	a original continua em `40/100`
B	espada `40/100`; madeira `2`	`false`; espada `40/100`; madeira `2`	a madeira termina em `1`

Em A, conferir só o retorno esconde o colaborador ignorado. Em B, conferir só a durabilidade esconde o consumo indevido.

Contratos mostram o que podemos solicitar

O **tipo declarado** determina as chamadas permitidas na compilação.

O **cast** permite outro uso compatível da referência, preservando o objeto. `instanceof` verifica a compatibilidade antes da conversão.

O **objeto alcançado** determina a implementação executada. Contratos distintos expressam capacidades distintas.

Delegar e verificar exige observar **o colaborador recebido**, o resultado e os estados envolvidos.

ATIVIDADE

APROFUNDAMENTO OPCIONAL — REATRIBUIÇÃO APÓS A SELEÇÃO

A variável local substitui o item na lista?

Inventário novo: apenas uma `Espada(40)`, na posição `0`.

```
Item selecionado = inventario.obter(0);
selecionado = new Escudo(30);
PilhaMaterial madeira = new PilhaMaterial("madeira", 2);
new Bigorna().tentarReparar(selecionado, madeira);
inventario.listar();
System.out.println(selecionado.descrever());
```

Quais são as duas linhas impressas? O escudo substituiu a espada no inventário?

APROFUNDAMENTO OPCIONAL — CONFERIR A REATRIBUIÇÃO LOCAL

A lista conserva a referência para a espada

```
Espada de ferro: 40/100
Escudo: 50/80
```

A reatribuição muda somente `selecionado`. O escudo não substitui a espada na lista.

A bigorna recebe e repara o escudo recém-criado. O inventário continua apresentando a espada, sem alteração.